

**САНКТ-ПЕТЕРБУРГСКИЙ НАЦИОНАЛЬНЫЙ
ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ ИТМО**

Дисциплина: Бэк-энд разработка

Отчет

Домашняя работа №3
«Тестирование API средствами Postman»

Выполнила:

Олейник Полина

Группа 2.1

Проверил:

Добряков Д. И.

Санкт-Петербург

2026 г.

Задача

Реализовать тестирование API средствами Postman; написать тесты внутри Postman.

Тема проекта: вариант 1 – приложение для бронирования столиков в ресторанах. Тестировалось REST API монолита из ЛР1 (<http://localhost:8000/api/v1>).

Ход работы

1. Принятое решение

Сделала две отдельные коллекции, а не одну, потому что у них разные задачи.

- «ЛР1 — 1. Сценарии» – комплексный сценарий по основному процессу приложения, как это прямо требует задание: один пользователь проходит путь от регистрации до брони и отзыва, а в конце – отдельный блок с проверкой ошибок. Эта коллекция создаёт данные (город, кухню, ресторан, столики, слоты, бронь) и оставляет их в окружении для второй коллекции.
- «ЛР1 — 2. По группам оренари» – по одному-два запроса на каждую группу эндпоинтов из спецификации Д32 (Авторизация, Рестораны, Столики, Слоты, Меню, Отзывы, Администраторы ресторана, Бронирования, Справочники, Свободные столики, Пользователи, Фотографии, Удаление), чтобы показать, что реализован весь API, а не только один сценарий.

Обе коллекции запускаются одна за другой в одном окружении: вторая использует токены, id города, кухни, ресторана и столиков, оставленные первой.

2. Окружение и повторяемость

Завела окружение «ЛР1 — Бронирование ресторанов (localhost)» с `baseUrl = http://localhost:8000/api/v1`, данными сидовых пользователей (`admin@example.com`, `owner@example.com`) и пустыми полями под `id` и токены, которые заполняются во время прогона. Токены (`adminToken`, `ownerToken`, `guestToken`) отметила как `secret`, чтобы они не показывались в интерфейсе Postman в открытом виде.

Чтобы коллекции можно было запускать повторно, не пересоздавая базу в ручную, использовала два приёма:

- email гостя и «одноразовых» сущностей собирается с `{{ $timestamp }}` (`guest.{{ $timestamp }}@example.com`), поэтому повторная регистрация никогда не упирается в 409 «email уже занят»;
- дата брони вычисляется в `pre-request script` на 7 дней вперёд от сегодняшнего дня, а не зашита в тело запроса:

```
const bookingDate = new Date();
bookingDate.setDate(bookingDate.getDate() + 7);
pm.environment.set("bookingDate", bookingDate.toISOString().slice(0, 10));
```

Без этого коллекция через несколько дней после написания начала бы падать на проверке «дата брони не должна быть в прошлом».

3. Коллекция «1. Сценарии»: 47 запросов, 6 папок

Папки идут по порядку одного сквозного сценария:

- 1. Вход и регистрация – регистрация гостя, вход администратора и владельца ресторана (через сидовые учётки – по `/auth/register` роль ADMIN получить нельзя), профили;
- 2. Управление рестораном – город, кухня, ресторан, назначение владельца администратором, столики, слоты, меню (от лица ADMIN и владельца);
- 3. Поиск и свободные столики – публичные запросы без токена: список городов и кухонь, поиск ресторана, его страница, меню, фото, свободные столики до брони;
- 4. Бронирование – создание брони, её карточка (с вложенными данными ресторана, столика и слота), история своих броней, свободные столики после брони (столлик A1 должен пропасть из списка), подтверждение брони владельцем;
- 5. Отзывы – отзыв гостя, проверка, что рейтинг ресторана пересчитался, изменение отзыва, повторный пересчёт рейтинга, список отзывов;
- 6. Проверка ошибок – 13 запросов на некорректные и нежелательные ситуации.

Пример теста на создание брони – проверяю не только код ответа, но и что бронь встала в статус PENDING:

```
pm.test("статус 201", function () {
  pm.response.to.have.status(201);
});
const jsonData = pm.response.json();
pm.environment.set("bookingId", jsonData.id);
pm.test("статус брони PENDING", function () {
  pm.expect(jsonData.status).to.eql("PENDING");
});
```

А следующий запрос проверяет, что после брони столик A1 действительно пропал из списка свободных на эту дату и слот:

```
pm.test("статус 200", function () {
  pm.response.to.have.status(200);
});
pm.test("столик A1 больше не свободен", function () {
  const ids = pm.response.json().tables.map(t => t.id);
  pm.expect(ids).to.not.include(Number(pm.environment.get("tableId")));
});
```

Раздел «Проверка ошибок» покрывает основные коды: 400 (гостей больше вместимости столика), 401 (профиль без токена), 403 (изменение чужого ресторана), 404 (несуществующий ресторан), 409 (повторная регистрация того же email, повторная бронь того же столика/слота/даты). Например, для повторной брони:

```
pm.test("статус 409", function () {
  pm.response.to.have.status(409);
});
```

4. Коллекция «2. По группам оренарі»: 50 запросов, 13 групп

Эта коллекция короче по тестам на запрос (в основном проверяется только код ответа), зато покрывает более широкий список операций – включая администраторов ресторана, удаление, фотографии – то, чего почти нет в первом сценарии.

Отдельная сложность: часть операций из первой коллекции нельзя протестировать по-настоящему (например, удаление ресторана, на который уже есть отзывы и брони, всегда должно возвращать 409, а не 204). Поэтому для демонстрации именно успешного удаления завела «одноразовые» сущности внутри этой же коллекции – например, отдельный ресторан без единой брони и отзыва:

```
{
  "name": "Одноразовый ресторан {{$timestamp}}",
  "city_id": {{$cityId}},
  "cuisine_id": {{$cuisineId}},
  "address": "Тестовый адрес, 2",
  "average_check": 500,
  "opening_time": "09:00",
  "closing_time": "21:00"
}
```

Его же в конце папки «Удаление» и удаляю, получая настоящий 204 – а рядом, для контраста, стоит запрос на удаление столика из первой коллекции, у которого есть бронь, и там ожидаемый ответ именно 409 (ON DELETE RESTRICT, как и было спроектировано в Д31):

```
pm.test("статус один из [409]", function () {  
  pm.expect([409]).to.include(pm.response.code);  
});
```

5. Что нашла при тестировании

Тестирование готового API вручную показало пару мест, которые стоит знать при защите – это не ошибки самих коллекций, а особенности реализации, которые я зафиксировала прямо в описании соответствующих запросов.

- **Ответ 401 не соответствует общему формату ошибок.**

`auth.middleware.ts` – обычный Express-middleware, а не часть `routing-controllers`, поэтому он отвечает `response.status(401).send({message})` напрямую, в обход `ErrorHandlerMiddleware`. В теле нет поля `statusCode`, хотя спецификация Д32 его требует у всех ошибок. Тест на это специально проверяет отсутствие поля, а не его значение.

```
pm.test("тело без statusCode – middleware отвечает напрямую, минуя error-handler",  
function () {  
  pm.expect(pm.response.json().statusCode).to.be.undefined;  
});
```

- **Подтверждение брони проверяется не через общий `restaurantAdminMiddleware`.** У пути `/bookings/{id}/status` в адресе нет `restaurantId`, поэтому обычный middleware не подходит – в `booking.controller.ts` для этого свой метод `assertCanAccess`, который сам находит ресторан через столик брони и проверяет, администратор ли этого ресторана вызывающий.
- **Зависимость между запросами внутри папки.** Например, запрос «Профиль гостя» сравнивает `email` из ответа с переменной `guestEmail`, а её задаёт запрос «Регистрация гостя» несколькими шагами раньше в той же папке. Пока коллекция выполняется через Collection Runner по порядку, всё сходится. Но если во время отладки запускать запросы вручную не по порядку

(например, повторно нажать «Профиль гостя», не повторяя перед этим регистрацию), переменные останутся от разных прогонов и тест на совпадение email упадёт – не из-за ошибки в API, а из-за нарушенного порядка запуска. Это стоит держать в голове при демонстрации: коллекцию нужно всегда прогонять целиком, через Run collection, а не отдельными запросами.

6. Проверка

Прогнала обе коллекции через Collection Runner в окружении «LP1 — Бронирование ресторанов (localhost)», сначала «1. Сценарии», затем в том же окружении «2. По группам оренаpi».

Результат: «1. Сценарии» – 47 запросов, 70 проверок, 0 упавших; «2. По группам оренаpi» – 50 запросов, 53 проверки, 0 упавших. Эти же коллекции позже, без единой правки, использовались для проверки, что публичный API не изменился после разделения на микросервисы (LP2) и после развёртывания на сервере (LP4) – там они дали те же самые числа.

Вывод

В ходе работы протестировала REST API из LP1 средствами Postman: собрала комплексный сценарий по основному процессу приложения (регистрация, поиск, бронирование, отзывы, проверка ошибок) в одной коллекции и покрыла остальные группы эндпоинтов во второй. Внутри запросов написала тесты на коды ответов, содержимое тела и побочные эффекты операций (столик пропадает из списка свободных после брони, рейтинг ресторана пересчитывается после отзыва). Сделала коллекции воспроизводимыми: уникальные email через `{{timestamp}}` и дата брони, вычисляемая от текущего дня, позволяют запускать их повторно в любой день без ручной подготовки базы. По ходу тестирования нашла особенности реализации (формат ответа 401, собственная проверка прав для подтверждения брони) и зафиксировала их прямо в описаниях запросов. Обе коллекции прошли без ошибок: 70 из 70 и 53 из 53 проверок, и позже без изменений использовались для проверки LP2 и LP4.